Groupe de travail applicatif

Le problème d'échange de reins et sa résolution

2022-2023

Auteurs :
— **SOULEYMAN Mahamat Youssouf**
— **BAILLET Benjamin**
— **DESIR Danyouse**
— **DUDOIGNON Héloïse**
— **MARION Emilie**
— **WANG CHEOU Darwin**

Professeur encadrant : Ayse Nur Arslan

Résumé

Le problème des échanges rénaux (KEP) consiste à trouver un ensemble (cycles et chaînes) d'échanges qui maximise le nombre de greffes pouvant être réalisées pour un ensemble de patients atteints de maladie rénale et leurs donneurs incompatibles. Dans ce projet, nous avons traité les instances ne contenant pas de donneur altruiste. Ainsi, nous avons proposé un modèle sous forme d'un programme linéaire en nombres entiers (PLNE). Pour la résolution de celui-ci, nous avons utilisé l'algorithme de génération de cycles pour générer l'ensemble des cycles. L'outil principal utilisé pour la résolution de notre modèle a été Pulp. Les résultats que nous avons trouvés ont été cohérents. En effet, nous avons eu l'occasion de vérifier que les cycles solutions du problème sont bien disjoints, en outre, que chaque patient n'est impliqué que dans au plus un échange. De plus, nous avons aussi pu modéliser et résoudre la relaxation du PLNE pour k = 2 (le nombre maximal de couples impliqués dans un cycle) et pour k = $+\infty$.

Ces relaxations nous ont permis de fixer une borne inférieure et une borne supérieure pour la valeur de la fonction objectif. Cependant, le problème devient complexe lorsque la taille des instances augmente, le temps résolution du modèle croît de manière exponentielle en raison de l'algorithme de génération des cycles. Cette remarque est également valable si l'on augmente la taille maximale des cycles. Toutefois, cet aspect de temps de calcul n'est pas réellement problématique dans la mesure où on se rend compte que l'on obtient le nombre de greffes maximales pour k = 3 en général, et au-delà, cette valeur reste quasiment constante.

Table des matières

1 Présentation du problème **4**
1.1 Contexte 4
1.2 Définition du problème 5
1.3 Schématisation d'un problème simple 5

2 Modélisation mathématique du problème **6**
2.1 Modèle pour k quelconque 6
2.2 Modèle pour k = 2 7
2.3 Modèle pour k = $+\infty$ 7

3 Exploitation des données des instances **8**
3.1 Présentation des fichiers d'instance 8
3.2 Parseur d'instances 8
3.3 Algorithme de génération de cycles 10

4 Résolution et vérification **11**
4.1 Résolution du modèle à l'aide de Pulp 11
4.2 Vérification du résultat 12
4.2.1 Checker 12
4.2.2 Problème de matching (k=2) 13
4.2.3 Problème de flot (k=$+\infty$) 13

5 Résultats numériques et interprétation **13**
5.1 Interprétation des résultats 14
5.1.1 Comportement du modèle en fonction de la taille des instances (en fonction du nombre de couples) 14
5.1.2 En fonction de k (la taille maximale des cycles) 16

6 Travail collaboratif **17**
6.0.1 Organisation du travail 17
6.0.2 Les outils utilisés 17
6.0.3 Les difficultés rencontrées 17

7 Conclusion **18**

8 Annexes **19**
8.1 Graphe obtenu à partir d'une instance de 16 couples 19
8.2 Tableaux obtenus à partir des différentes instances 19
8.3 Lien vers nos codes 21

1 Présentation du problème

1.1 Contexte

L'insuffisance rénale chronique (IRC) est une maladie grave et irréversible d'apparition lente qui touche de nombreuses personnes dans le monde. Selon le rapport "Ma maladie chronique 2022" publié par la Société Francophone de Néphrologie Dialyse et Transplantation, cette maladie touche environ 3 millions de personnes en France. L'insuffisance rénale chronique est généralement la conséquence du diabète, de l'hypertension artérielle ou de dérèglements spontanés du système immunitaire. Cette maladie est une diminution progressive et irréversible des fonctions des reins qui ne filtrent plus correctement le sang de l'organisme. En 2016, l'étude "Global Burden of Disease Study" identifie l'IRC comme la 11ème cause la plus fréquente de décès dans le monde, raison pour laquelle nous nous intéressons à cette maladie et à ses solutions. Les coûts liés à l'insuffisance rénale avoisinent les 3 milliards d'euros pour l'année 2022. Pour résoudre ce problème, il y a deux solutions possibles qui sont la dialyse ou bien la greffe rénale :

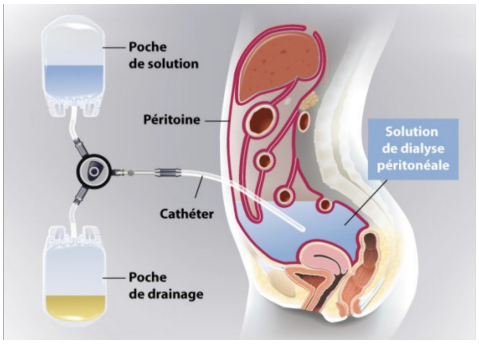


FIGURE 1 – Dialyse

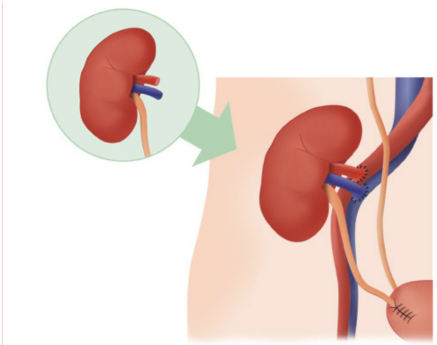


FIGURE 2 – Greffe

Bien que la dialyse soit plus facile à mettre en place, elle ne peut compenser que partiellement la dysfonction rénale. De plus, elle est très coûteuse sur le long terme et diminue de manière significative la qualité de vie du patient. En effet, le patient doit se rendre dans un centre hospitalier trois fois par semaine. De ce fait, nous nous intéressons à la greffe rénale, qui, une fois réalisée avec succès, permet au patient d'avoir une meilleure qualité de vie. Selon un communiqué de presse de l'Agence de la Biomédecine (ABM) datant de 2021, 3251 greffes rénales ont été réalisées dont 502 à partir de donneurs vivants. Généralement, les greffes de reins sont réalisées à partir d'un donneur décédé, grâce à l'avancée des législations, elles peuvent se faire à partir d'un donneur vivant, suite à des tests de compatibilité entre le patient et le donneur. Si ce test est positif, la greffe peut avoir lieu, sinon nous pouvons avoir recours au programme d'échange de reins (KEP) :

Il s'agit d'un système centralisé dans lequel on a un ensemble de couples patient-donneur incompatibles. Ce programme permet d'effectuer des croisements tout en recherchant la compatibilité entre les couples patient-donneur pour une greffe provenant des autres donneurs inscrits dans le programme.

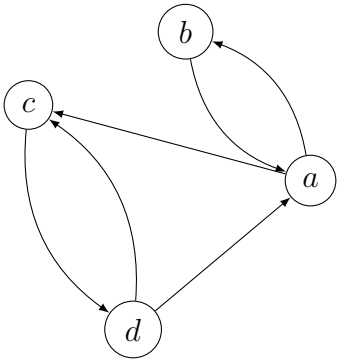
Nous allons utiliser la programmation linéaire en nombres entiers pour la modélisation mathématique du problème, ainsi que certaines bibliothèques open source (Pulp) pour sa résolution. Pour ce faire, nous programmerons différents algorithmes en Python afin d'exploiter les données issues des différentes instances fournies.

1.2 Définition du problème

Nous définissons le problème d'échange rénal (KEP) de la manière suivante : celui-ci consiste à trouver un chemin optimal (ici de poids maximal) d'échanges rénaux dans un ensemble de couples donneur-patient. Définissons un graphe orienté, appelé graphe de compatibilité, G = (V = P ∪ N, A), avec une fonction de poids sur les arcs w : A → R^+. Les sommets P sont les couples donneurs-patients incompatibles, les sommets N sont les donneurs altruistes. Nous ajoutons un arc pondéré entre les sommets u et v si le donneur u peut donner son rein au patient v. Donc w(uv) est le bénéfice médical de la greffe. Ainsi, une solution de KEP est un ensemble de cycles et chaînes disjointes de poids maximum dans le graphique G (notons que les cycles et les chaînes doivent être disjoints car un donneur ne peut donner qu'une seule fois et un patient doit recevoir un seul rein). De plus, un cycle de dons doit avoir une taille limitée k puisqu'il implique 2k opérations chirurgicales simultanées. Une chaîne de dons peut avoir une taille limitée l, mais aucun consensus n'existe sur ce paramètre. Donc, résoudre le KEP équivaut à résoudre un PLNE, qui, comme nous l'avons vu, consiste à trouver des cycles disjoints de taille au plus k et des chaînes de taille au plus l comme poids maximal.

1.3 Schématisation d'un problème simple

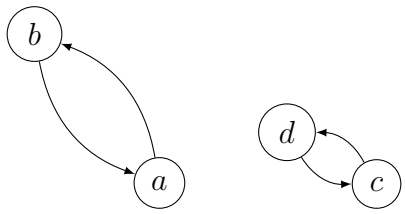
Afin de représenter le problème d'échange de reins également appelé KEP (Kidney Exchange Problem), nous pouvons prendre comme exemple un graphe orienté de 4 sommets où chaque arête représente une greffe et où chaque sommet représente une paire patient-donneur qui est incompatible. La flèche de l'arête permet d'indiquer qu'un donneur pointe vers un receveur potentiel. La flèche indique donc le sens de la greffe. Le premier graphe est le graphe initial et les deux graphes qui suivent sont les solutions du problème.



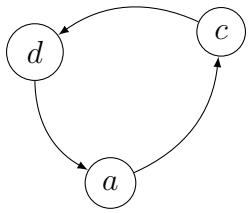
Données :
— (a) peut donner à (b) et (c) ;
— (b) peut donner à (a) ;
— (c) peut donner à (d) ;
— (d) peut donner à (a) et (c) ;

Solutions réalisables

Solution 1 :



Solution 2 :



2 Modélisation mathématique du problème

2.1 Modèle pour k quelconque

Pour modéliser le problème d'échange de reins aussi appelé KEP, nous avons fait appel à nos connaissances sur la PLNE (Programmation Linéaire en Nombres Entiers). Il existe plusieurs modélisations possibles afin de résoudre le problème d'échange de reins et nous allons donc étudier un premier modèle. Dans un premier temps, on considère la formulation en cycles sans les donneurs altruistes (donc il n'y a pas de chaines) : une des manières de modéliser le problème se fait de la façon suivante. Etant donné l'ensemble de tous les cycles possibles de taille au plus k, nous devons trouver l'ensemble des cycles C^* qui maximise le poids total avec la containte suivante, chaque couple doit être choisi dans au plus un cycle. Dans cette formulation basée sur des échanges, chaque cycle est associé à une variable binaire indiquant si il est choisi ou non dans la solution. Cela conduit à un modèle simple mais ayant un nombre exponentiel de variables .

Soit :

— U_k L'ensemble des cycles de taille inférieure ou égale à k ;
— U_i L'ensemble des cycles contenant le couple i avec $i \in I$;
— I L'ensemble des couples patient-donneur ;
— Les données recueillies sont les $w_{u \in U_k}$ représentant le poids de chaque cycle u qui est équivalent à la somme des poids des arêtes ;

On veut donc maximiser le poids avec les greffes choisies ici :

$$\max \sum_{u \in U_k} w_u x_u \tag{1}$$

$$\text{s.t.} \sum_{u \in U_i} x_u \leq 1 \qquad \forall i \in I \tag{2}$$

$$x_u = \begin{cases} 1 & \text{si le cycle } u \text{ est choisi pour la solution} \\ 0 & \text{sinon} \end{cases} \tag{3}$$

2.2 Modèle pour k = 2

Dans ce modèle on fixe la taille maximale de cycles d'échanges à k = 2. Le problème consiste donc à trouver un ensemble de couple d'échanges qui maximise le poids total.

Soit :

— I L'ensemble des couples patient-donneur (les sommets du graphe) ;
— E L'ensemble des arêtes du graphe formé par les echanges ;
— δ_i L'ensemble des arêtes incidentes au somment i avec $i \in I$;
— $w_{e \in E}$ le poids de l'arête e ;

$$\max \sum_{e \in E} w_e x_e \tag{4}$$

$$\text{s.t.} \sum_{e \in \delta_i} x_e \leq 1 \qquad \forall i \in I \tag{5}$$

$$x_e = \begin{cases} 1 & \text{si l'arête } e \text{ est choisie pour la solution} \\ 0 & \text{sinon} \end{cases} \tag{6}$$

2.3 Modèle pour k = $+\infty$

Cette formulation sera utilisée par la suite pour le problème de flot.

Soit :

— I L'ensemble des couples patient-donneur (les sommets du graphe) ;
— E L'ensemble des arêtes du graphe formé par les échanges ;
— R_i L'ensemble des sommets voisins du sommet i dont l'arête rentre dans $i \in I$;
— S_i L'ensemble des sommets voisins du sommet i dont l'arête sort de $i \in I$;
— $w_{e \in E}$ le poids de l'arête e ;

$$\max \sum_{(i,j) \in E} w_{ij} x_{ij} \tag{7}$$

$$\text{s.t.} \sum_{j \in R_i} x_{ji} \leq 1 \qquad \forall i \in I \tag{8}$$

$$\text{s.t.} \sum_{j \in R_i} x_{ji} = \sum_{j \in S_i} x_{ji} \qquad \forall i \in I \tag{9}$$

$$x_{ij} = \begin{cases} 1 & \text{si l'arête } (i,j) \text{ est choisie pour la solution} \\ 0 & \text{sinon} \end{cases} \tag{10}$$

3 Exploitation des données des instances

3.1 Présentation des fichiers d'instance

Nous avons à disposition un ensemble d'instances que l'on peut retrouver sur le lien suivant "https ://www.preflib.org/dataset/00036". Ces instances sont générées sur la base de statistiques médicales. Nous disposons de 10 instances ".wmd" avec chacune 16, 32, 64, 128, 256, 512, 1024 et 2048 patients sans donneurs altruistes. Il y a également des instances ".dat" mais nous ne les utilisons pas ici. Il est important de noter qu'au cours de notre étude, nous n'allons pas utiliser les instances comportant des donneurs altruistes.

Dans un graphe, chaque arête a une source (le donneur) et plusieurs destinations (les patients qui nécessitent une greffe). Une greffe est donc représentée par une arête orientée. Toutes les arêtes ont un poids de 1 ici, mais nous avons adapté nos algorithmes de telle sorte qu'il puissent prendre en considération des poids différents de 1. Nous pouvons constater qu'en fin de page de l'image de la section 3.2, nous avons 3 colonnes, la première colonne représente les patients, la seconde les donneurs et la troisième le poids de chaque greffe.

```
# FILE NAME: 00036-00000001.wmd
# TITLE: Kidney Matching - 16 with 0
# DESCRIPTION:
# DATA TYPE: wmd
# MODIFICATION TYPE: synthetic
# RELATES TO:
# RELATED FILES: 00036-00000001.dat
# PUBLICATION DATE: 2013-08-17
# MODIFICATION DATE: 2022-09-16
# NUMBER ALTERNATIVES: 16
# NUMBER EDGES: 59
# ALTERNATIVE NAME 1: Pair 1
# ALTERNATIVE NAME 2: Pair 2
# ALTERNATIVE NAME 3: Pair 3
# ALTERNATIVE NAME 4: Pair 4
# ALTERNATIVE NAME 5: Pair 5
# ALTERNATIVE NAME 6: Pair 6
# ALTERNATIVE NAME 7: Pair 7
# ALTERNATIVE NAME 8: Pair 8
# ALTERNATIVE NAME 9: Pair 9
# ALTERNATIVE NAME 10: Pair 10
# ALTERNATIVE NAME 11: Pair 11
# ALTERNATIVE NAME 12: Pair 12
# ALTERNATIVE NAME 13: Pair 13
# ALTERNATIVE NAME 14: Pair 14
# ALTERNATIVE NAME 15: Pair 15
# ALTERNATIVE NAME 16: Pair 16
1,5,1.0
1,6,1.0
2,1,1.0
2,3,1.0
2,11,1.0
2,12,1.0
```

FIGURE 3 – Exemple de fichier ".wmd"

3.2 Parseur d'instances

Afin de résoudre le problème d'échange de reins, nous avons besoin de récupérer les données des fichiers d'instances ".wmd" afin de les traiter avec nos différents algorithmes. Pour cela nous avons créé un parseur d'instances qui récupére le nombre d'arêtes ainsi que le nombre de sommets (qui est donc le nombre de couples patient-donneur du graphe). A la suite du parseur d'instances nous créons un dictionnaire qui est en quelque sorte la matrice d'adjacence du graphe dont les sommets sont les couples patient-donneurs et les arêtes sont les greffes. Dans ce dictionnaire on associe à chaque

sommet les sommets voisins vers lesquels il pointe (autrement dit pour un donneur on peut avoir plusieurs receveurs). Le dictionnaire est sous la forme suivante : 1 : [3, 16], 2 : [4, 14] (ici le donneur du couple 1 peut donner au receveur du couple 3 et du couple 13). Voici le pseudo-code du parseur d'instances que nous avons réalisé, avec la création du dictionnaire :

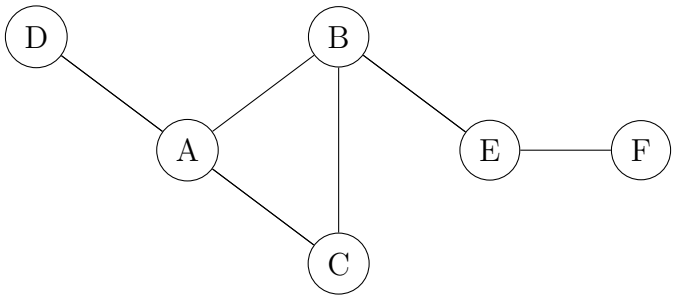
Algorithm 1: Pseudo-code du parseur d'instances

```
Entrée: nom du fichier
lineCount, nbArete, nbCouple, nbLignes ← 0 ;
listeAretes ← liste vide ;
Classe  arete
    Init(self, s) :
        self.donneur ← le premier élément de s;
        self.patient ← le deuxième élément de s;
        self.poid ← le troisième élément de s;
Ouvrir le fichier filename ;
while lecture de la ligne suivante do
    if la ligne contient "NUMBER ALTERNATIVES" then
        Afficher lineCount ;
        nbCouple ← "NUMBER ALTERNATIVES" ;
    else
        if la ligne contient "NUMBER EDGES" then
            nbArete ← "NUMBER EDGES" ;
            nbLignes ← lineCount ;
        else
            if (lineCount > nbLignes + nbCouple) et (lineCount > lineCount + 1 -
             (nbLignes + nbCouple)) then
                A ← new Arete(x) ;
                Ajouter A à listeArete ;
            else
                –
    lineCount ← lineCount + 1 ;
Afficher "Nombre de paires patient-donneur : ", nbCouple ;
Afficher "Nombre d'arêtes du graphe : ", nbArete ;
for k de 0 à nbArete- 1 do
    Afficher "Donneur : ", listeArete [k].donneur ;
    Afficher "Patient : ", listeArete [k].patient ;
    Afficher "Poids : ", listeArete [k].poid ;
dicoGraphe = ensemble vide
list = liste vide
for k de 1 à nbCouple do
    list = liste vide
    for i de 0 à longueur(listeArete)-1 do
        Si le donneur du ème élément de listeArete = k : On ajoute à list
    Le kième élément de dicoGraphe= list
Afficher(dicoGraphe)
```

3.3 Algorithme de génération de cycles

Nous avons ensuite besoin de générer des cycles pour mettre en place le problème d'échange de reins KEP entre les différents couples patient-donneur incompatibles que l'on a récupérés avec le parseur d'instances. Pour écrire cet algorithme nous avons utilisé la méthode du parcours en profondeur qui consiste à parcourir tous les sommets d'un graphe. Nous allons alors schématiser cette méthode ci-dessous à l'aide d'un exemple simple :

Graphe considéré



Prenons pour exemple un graphe non orienté. On part du sommet A pour aller dans l'ordre logique, car c'est la première lettre de l'alphabet (on ajoute A à la pile), puis on va à B (on ajoute B à la pile), puis on va à C (on ajoute C à la pile). Ensuite, on dépile vers B car les voisins de C sont déja visités, on va à E (on ajoute E à la pile) puis on va à F (on ajoute F à la pile). Pour continuer on dépile vers E car les voisins de F sont déja visités, puis on dépile vers B car les voisins de B sont déja visités, on dépile vers A car les voisins de A sont déja visités. Pour finir on va à F (on ajoute F à la pile). Finalement on a tous les sommets, donc le graphe obtenu est connexe. Cela achève le parcours en profondeur.

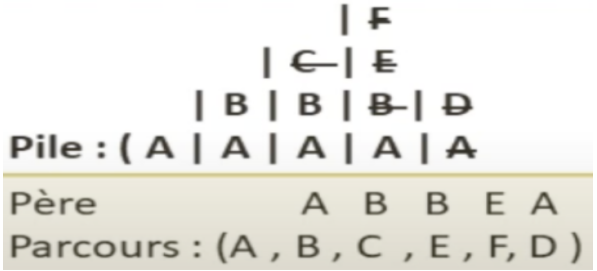


FIGURE 4 – La pile

Ensuite nous créons une classe Cycle avec deux fonctions sous-jacentes qui seront représentés ci-dessous en pseudo-code. L'algorithme de génération de cycles se base sur le principe du parcours en profondeur et il utilise la classe Cycle. On donne donc le pseudo-code d'une partie de l'algorithme de parcours en profondeur, utilisant la classe Cycle. Cette classe contient trois éléments (Cycle.cycle : la liste des sommets du cycle, Cycle.taille : donnant la taille du cycle et Cycle.poids : donnant le poids du cycle).

Algorithm 2: Pseudo-code du parcours en profondeur

```
Classe  Cycle
  Init(self, list)
    self.cycle ← liste du cycle ;
    self.taille ← longueur de la liste;
  https ://fr.overleaf.com/project/63e1fdeb99fb050f9930a017
    self.poids ← somme poids des arêtes ;

ParcoursEnProfondeur (graphe, noeud, visites, pile, cycles, tailleMaxCycle)
visites[noeud] ← True ;
Ajouter noeud à pile;
if Taille de pile < tailleMaxCycle +1 then
    for voisin dans graphe[noeud] do
      if Voisin = pile[0] then
        cycle = pile
          C = Cycle(cycle);
          Ajoute C à Cycles;
else
  if visites[voisin] = False then
    ParcoursEnProfondeur (graphe, voisin, visites,
    pile, cycles, tailleMaxCycle);
    visites[pile[-1] = False;
    Supprime pile[-1];

TrouverCycles (graphe, tailleMaxCycle)
  Initialiser un dictionnaire visites : tout noeud du graphe associé à False ;
  Cycles = liste vide;
  NoeudDépartVisite = liste vide;
  for chaque noeud dans le graphe do
    Ajoute noeud à NoeudDépartVisite
    pile = liste vide
    ParcoursEnProfondeur(graphe, noeud, visites, pile, cycles, taille_max_cycle)
      for chaque noeud dans le graphe do
        if noeud est dans NoeudDépartVisite then
          visite[noeud] = True
        else
          visite[noeud] = False
retourner(cycles, longueur(cycles))
```

4 Résolution et vérification

4.1 Résolution du modèle à l'aide de Pulp

Afin de résoudre le modèle mathématique, nous avons utilisé la bibliothèque Pulp. Nous avons fait en sorte que le programme affiche le statut de la solution (Réalisable,

Irréalisable, Non borné ou Optimal), le temps de calcul, la valeur de la fonction objectif ainsi que le(s) cycle(s) solution(s). Nous utilisons ici encore une fois la classe Cycle. Ci-dessous nous retrouvons une partie du code que nous avons réalisé en se basant sur l'instance "00036-00000010.wmd" :

Algorithm 3: Pseudo-code de la résolution du modèle avec Pulp

```
C, nbC= trouverCycles(dicoGraphe, 3) ;
prob = LpProblem('Modèle pour k quelconque', LpMaximize) ;
x = LpVariable.dicts(name='x', indices = range(len(C)), cat='Binary') ;
prob += lpSum([x[u] * C[u].poids for u in range(len(C))]) ;
for i de 0 à nbCouple - 1 do
    Ui = [] ;
    for u de 0 à len(C) - 1 do
        if i+1 est dans C[u].getSommets() then
            Ui.append(u)
    prob += lpSum([x[u] for u in Ui]) <= 1

prob.solve(PULP(timeLimit=30, threads=1))
```

Le résultat est le suivant :
- Status de la solution : Optimal
- Temps de résolution (s) : 0.009364128112792969
- Valeur de la fonction objectif (poids total) : 4.0
- Cycles solutions : Cycle 1 : [4, 6] de poids 2.0 et Cycle 4 : [14, 15] de poids 2.0

Cela nous amène donc à la partie suivante et celle-ci portera sur la vérification des résultats obtenus dans différents cas.

4.2 Vérification du résultat

Grâce à la résolution de notre problème de PLNE, nous obtenons une solution qui est un ensemble de cycles ainsi que la valeur de la solution qui est la somme des poids des cycles solutions. Nous allons donc étudier la solution afin de vérifier qu'elle est juste et qu'elle correspond bien à tous les critères, ce qui va être l'objet de cette partie.

4.2.1 Checker

Nous allons dans un premier temps utiliser un Checker. Il s'agit d'un programme informatique qui va nous permettre de vérifier que notre solution est bien réalisable. Il va vérifier 2 conditions : Premièrement, si la solution est bien un ensemble de cycles disjoints. Deuxièmement, si les cycles sont bien de taille inférieure ou égale à k. Après avoir vérifié ces 2 conditions il va afficher à l'écran si la solution est réalisable ou non. Il est important de noter que ces 2 conditions ne suffisent pas pour dire que la solution du PLNE est bien la bonne. En effet nous allons désormais vérifier la valeur de la solution objectif en l'encadrant avec une borne inférieure et supérieure, ce qui sera l'objet des 2 sous-parties suivantes. Nous allons utiliser le problème de matching pour trouver une borne inférieure et le problème de flot pour trouver une borne supérieure. Puis nous allons vérifier si la solution du PLNE est bien comprise entre ces deux bornes.

4.2.2 Problème de matching (k=2)

Etant donné qu'il serait quelque peu fastidieux de créer un programme qui vérifie les résultats pour chaque taille de cycles, nous nous sommes dans un premier temps concentrés sur le problème de matching, avec des cycles de taille 2. C'est un problème de couplage maximum dans un graphe non orienté. Ainsi le résultat obtenu nous permettra d'avoir une borne inférieure de la solution du PLNE. Ce problème de matching est assez similaire à celui de la résolution en PLNE, cependant, au lieu d'utiliser notre algorithme de génération de cycles, nous nous sommes limités à prendre en argument la liste de tous les cycles de taille 2 obtenus à partir de notre instance dans le but d'obtenir un graphe non-orienté. De ce fait, ici nous nous sommes basés sur le modèle pour k=2 de la page 4 (section 2.2). En effet, nous avons par la suite créé notre variable de décision qui prend comme valeur 1 si l'arête e est prise dans le cycle et 0 sinon, ses indices sont parcourus dans notre liste des delta(i). La resolution de ce modèle résout le KEP pour k = 2. Notons que nous aurions pu, au lieu d'utiliser Pulp pour le problème de matching, utiliser l'algorithme d'Edmonds Karp, qui se base sur la notion de parcours en largeur et de chemin augmentant. Dans ce cas le KEP peut être résolu polynomialement via l'algorithme d'Edmonds.

4.2.3 Problème de flot (k=$+\infty$)

Toujours dans l'optique de vérifier la solution en PLNE, il impératif de maximiser ces résultats avec une méthode assez simple, et s'assurer que les résultats obtenus sont toujours inférieurs ou égaux à cette maximisation. Pour cela, la méthode du flot a été utilisée. Pour imager cette méthode, on ne met pas de contrainte sur la longueur des cycles, et on cherche le nombre maximal de greffes possible. Dans ce cas, on n'a pas besoin d'utiliser un algorithme de génération de cycles. En effet, une méthode d'optimisation en nombre entier, à partir d'une liste des arêtes (modélisant les greffes possibles) suffit. De ce fait, ici nous nous sommes basés sur le modèle pour k infini de la page 4 (section 2.3). On utilise différentes contraintes afin de résoudre ce problème. Tout d'abord, nous voulons obtenir des cycles. Pour cela, il faut une contrainte qui oblige que le flot entrant sur un sommet soit le même en sortie de ce sommet, pour tout sommet du graphe. De manière plus concrète, prenons un sommet i du graphe solution, s'il y a une arête partant du sommet j et allant au sommet i, alors il y aura forcément une arête partant du sommet i et allant vers un autre sommet. De la même manière s'il n'y a pas d'arête entrante il n'y aura pas d'arête sortante.

De plus, nous voulons que chaque sommet n'apparaisse pas plus d'une fois dans les cycles solutions. Pour cela, on veut, pour chaque sommet, qu'il y ait au plus une arête arrivant sur le sommet, et au plus une arête sortant de ce sommet.

En résolvant ce problème avec pulp, on obtient une solution qui prend le maximum de cycles possible, et le poids maximal obtenu avec ces cycles, tout en respectant la contrainte qu'un sommet ne soit contenue dans au plus un cycle. Ensuite on compare ce résultat à celui de notre problème principal qui doit forcément avoir un poids solution inférieur à celui du flot. Cela a en effet été le cas pour notre résolution.

5 Résultats numériques et interprétation

Dans cette partie nous allons utiliser les algorithmes mis en place pour résoudre le modèle avec différentes instances. Cela nous permettra de génerer un ensemble de

données numériques. Ces données seront interprétées pour nuancer la complexité algorithmique du modèle.

5.1 Interprétation des résultats

Nous allons étudier différents graphiques que nous avons obtenus à l'aide de nos tableaux.

5.1.1 Comportement du modèle en fonction de la taille des instances (en fonction du nombre de couples)

Dans cette partie nous allons prouver avec les données générées que la résolution de notre modèle avec l'algorithme de génération de cycles a une complexité exponentielle en temps et en espaces pour les grandes instances. Nous avons fixé la taille maximale de cycles à deux pour des raisons techniques. Pour chaque nombre de couples, nous avons utilisé 10 différentes instances pour calculer le nombre de cycles , la valeur du poids maximum et le temps de résolution du PLNE de chacune des instances. Ensuite nous avons fait la moyenne arithmétique de ces données pour avoir un nombre de cycles, la valeur maximum de la fonction objectif et un temps de résolution pour chaque nombre de couples. Ces données nous ont permis de tracer la courbe de l'évolution du nombre de cycles (figure 6), de la valeur maximale de la fonction objectif (figure 5) et le temps de résolution (figure 7) en fonction du nombre de paires patient/donneur.

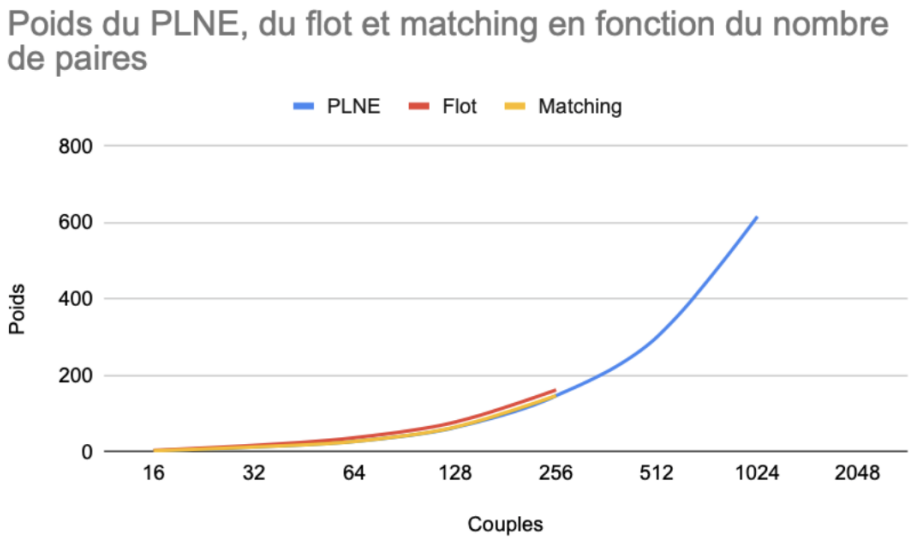


FIGURE 5 – Valeur de la solution obtenue avec le PLNE, le matching et le flot en fonction de la taille des instances

Le graphique ci-dessus montre que la valeur de la solution du problème de PLNE est bien encadrée par sa borne inférieure et sa borne supérieure. Sa borne inférieure étant obtenue grâce à la résolution du problème de matching (pour k=2), et sa borne inférieure étant obtenue grâce à la résolution du problème de flot (pour k infini).

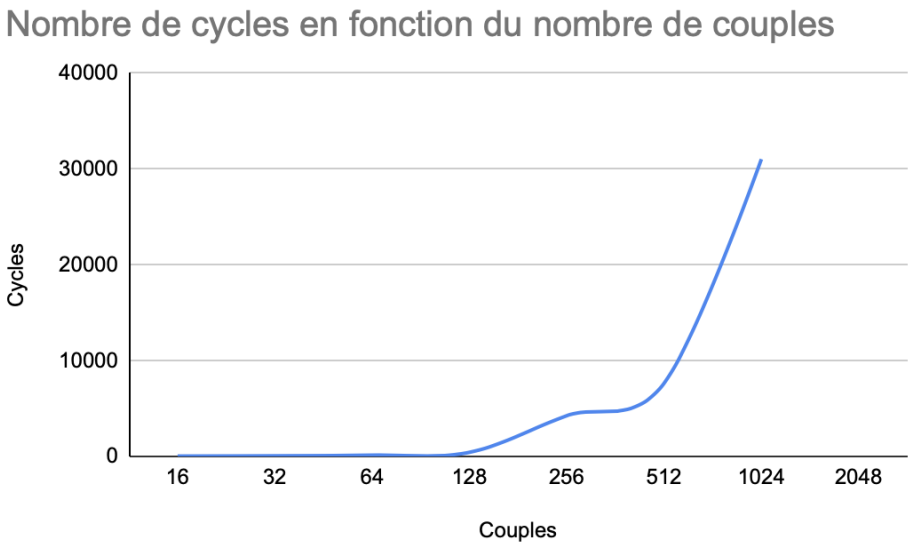


FIGURE 6 – Nombre de cycles générés en fonction de la taille des instances

Le graphique ci-dessus montre l'évolution du nombre de cycles obtenus grâce à notre algorithme de génération de cycles en fonction de la taille des instances. On remarque que le nombre de cycles augmente progressivement jusqu'à 128 couples, puis jusqu'à 512 couples, avant d'augmenter de manière exponentielle. En effet, ici on augmente la taille des instances et donc le nombre de couples patient-donneur incompatibles. Ceci explique que la résolution du modèle avec l'algorithme de génération de cycles a une complexité exponentielle en espace.

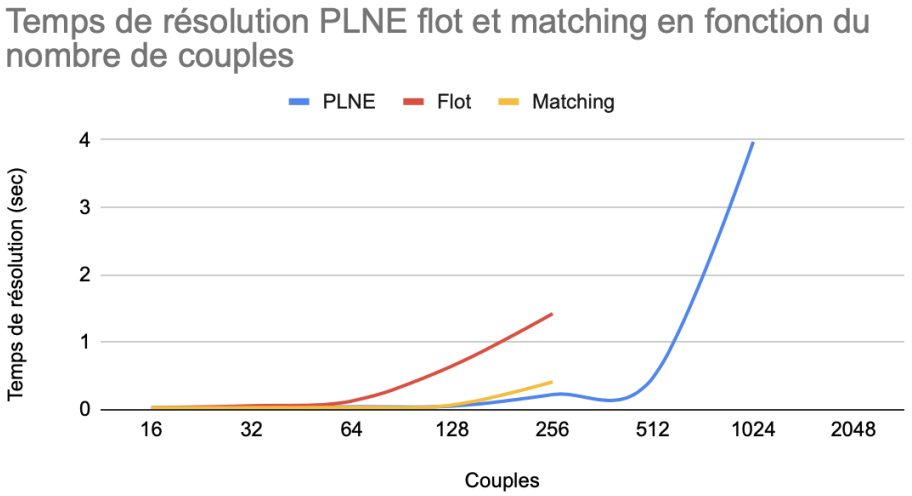


FIGURE 7 – Temps de résolution du PLNE, du matching et du flot en fonction de la taille des instances

Ce graphique nous montre que le temps de résolution augmente pour le problème de flot à partir de 64 couples. Pour le problème de matching il augmente à partir de 128 couples. Pour la résolution du PLNE, le temps de résolution augmente exponentiellement à partir de 512 couples. Donc plus on augmente la taille des instances plus le temps de résolution du PLNE avec l'algorithme de génération de cycles augmente de façon exponentielle. C'est pourquoi cet algorithme a une complexité exponentielle en

temps.

5.1.2 En fonction de k (la taille maximale des cycles)

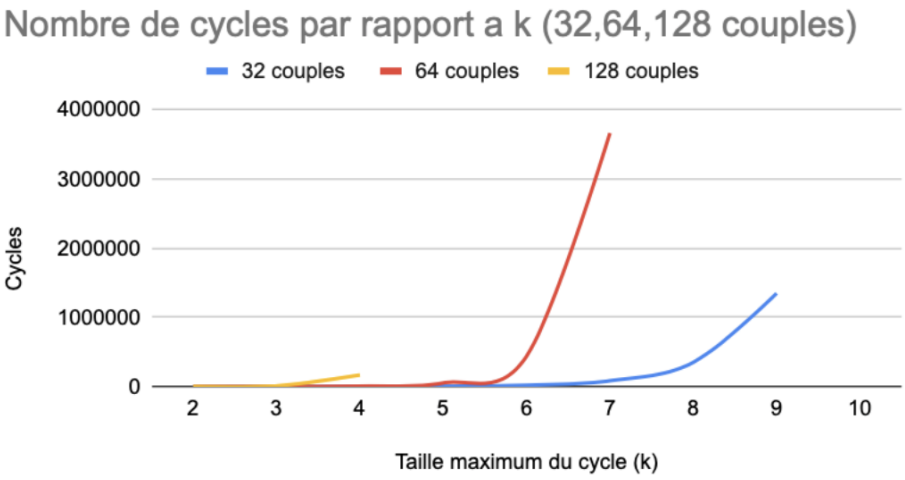


FIGURE 8 – Nombres de cycles générés en fonction de k

Ce graphique nous montre que le nombre de cycles générés avec l'algorithme de génération de cycles augmente rapidement lorsqu'on augmente k, la taille maximale des cycles. La courbe pour 128 couples n'est pas terminée car cela prenait trop de temps de calcul.

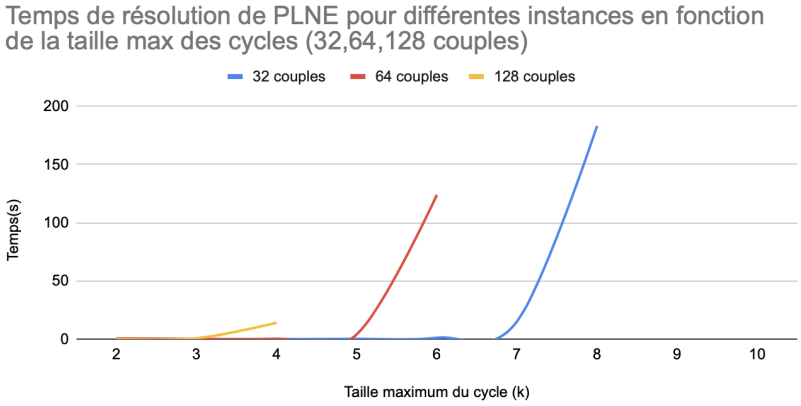


FIGURE 9 – Temps de résolution du PLNE en fonction de k

Ce graphique nous montre que le temps de résolution du PLNE augmente rapidement lorsqu'on augmente k, la taille maximale des cycles. La courbe pour 128 couples n'est pas terminée pour les même raisons que pour le graphique qui précède. On constate d'ailleurs que le graphique ressemble au précédent, ce qui est logique.

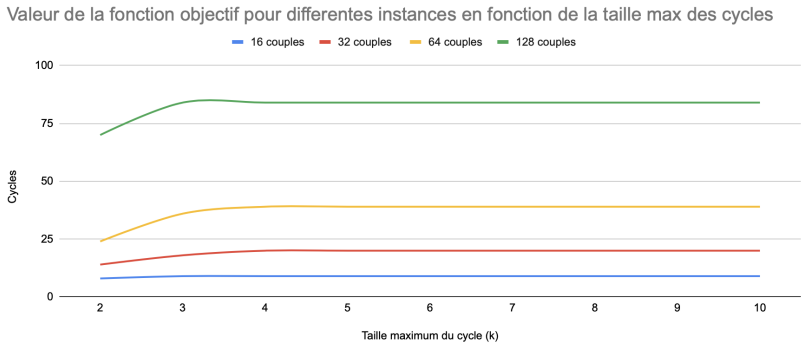


FIGURE 10 – Valeur de la fonction objectif en fonction de k

Ce graphique nous montre que la valeur de la fonction objectif est à peu près constante lorsqu'on augmente la valeur de k pour différentes instances. En effet, il y a une légère augmentation au début puis ça reste constant.

6 Travail collaboratif

6.0.1 Organisation du travail

Pour poursuivre, durant tout le semestre nous avons mis en place plusieurs rendez-vous hebdomadaires dont un avec notre encadrante Mme Ayse Nur Arslan afin, dans un premier temps de faire un point sur l'avancement de notre projet et des objectifs qu'il restait à accomplir. Cela passait par une mise en commun du travail que chacun avait au préalable effectué en autonomie et le partage de chacune de nos idées. La majeure partie du travail que nous avons effectué a été faite en amont de chaque rendez-vous à l'aide de plusieurs outils et logiciels qui seront détaillés par la suite.

6.0.2 Les outils utilisés

Afin de remplir nos objectifs, nous devions utiliser différents outils qui nous ont servi tout au long du semestre. Cinq plateformes avaient particulièrement grand intérêt pour nous. Premièrement, afin de communiquer et de partager nos idées ainsi que les informations nécessaires à notre progression, nous avons utilisé la plateforme de communication Whatsapp, simple d'utilisation, elle nous a permis de faire des résumés des réunions avec notre encadrante du projet. Par la suite, il nous fallait aussi un moyen de stocker tous nos algorithmes avec un accès facile et immédiat, c'est pour cela que nous avons utilisé Google Collab. Nous nous sommes aussi aidé de Google Docs, Google Sheets, logiciel avec lequel nous avons fait des tableaux pour nous répartir le travail de manière équitable et faire état de l'avancement de notre projet. L'un des logiciels les plus utilisé a été Overleaf, celui-ci nous a notamment permis d'écrire notre rapport dans un langage mathématique précis et compréhensible. Pour finir, nous avons utilisé le logiciel Canva afin de créer notre diaporama pour notre soutenance.

6.0.3 Les difficultés rencontrées

Bien qu'une fois lancés dans notre projet nous savions tout ce que nous avions à faire, nous avons tout de même rencontré plusieurs difficultés au cours du semestre. En effet, les débuts ont été quelque peu ardus, bien avant de commencer quoi que ce

soit, nous devions prendre notre temps afin de bien comprendre le problème est savoir qu'est ce qui était attendu de nous. Une fois les objectifs établis, nous devions nous répartir le travail en fonction de l'affinité de chaque personne dans chaque domaine à traiter. Une partie d'entre nous devait réaliser un travail de modélisation assez complexe afin de comprendre comment on aborderait le problème, ce que nous n'avions pas fait depuis un bon moment, tandis que l'autre partie devait se charger de commencer à faire des algorithmes en Python, langage que nous n'avions pas utilisé depuis plus d'un an. Ensuite, nous avons eu du mal à écrire l'algorithme de génération de cycles et à résoudre le problème de flot.

7 Conclusion

Pour conclure, ce programme d'échange de reins est très important, dans la mesure où l'insuffisance rénale est la 11ème cause de mortalité dans le monde. Il existe de nombreux couples patient-donneur incompatibles, de ce fait, il est essentiel de trouver des donneurs aux patients de ces couples. En effet, tout au long de notre travail, nous avons pu constater que ce programme augmentait fortement la possibilité de greffes. Grâce à celui-ci, de nombreuses vies pourraient être sauvées. Après avoir modélisé le problème mathématiquement, nous avons dans un premier temps récupéré les données des instances à l'aide de notre parseur, par la suite, nous avons généré les cycles de chaque instance. Les résultats que nous avons trouvés ont été cohérents. En effet, l'ensembles des données générées nous ont permis de constater la complexité du problème en temps et en espace. De ce fait, notre modèle ne représente pas la meilleure solution pours des instances de grandes tailles. Ainsi, plusieurs modèles de résolution ont été proposés et étudiés par les chercheurs.

Nous avons tout d'abord l'algorithme de programmation linéaire en nombres entiers : celui-ci utilise la théorie des graphes pour trouver une solution optimale au problème des échanges rénaux. Il a une complexité exponentielle en temps et en espace, ce qui le rend inefficace pour les instances de grande taille.

Nous avons également l'algorithme d'optimisation combinatoire : Cet algorithme est une méthode heuristique qui utilise des techniques de recherche locale pour trouver une solution proche de l'optimum global. Cette méthode est souvent utilisée en pratique pour résoudre le problème des échanges rénaux en raison de sa rapidité et de son efficacité. La complexité de cet algorithme dépend de la taille de l'instance, mais il est souvent possible d'obtenir une solution en temps polynomial. Il est important de noter que chaque algorithme a ses avantages et ses inconvénients en termes de précision et de rapidité. Les choix d'algorithmes dépendent souvent de la taille de l'instance et des contraintes spécifiques du problème des échanges rénaux.

Nous tenons à remercier Mme Ayse Nur Arslan, notre professeure encadrante, qui nous a été d'une aide précieuse pour ce projet.

8 Annexes

8.1 Graphe obtenu à partir d'une instance de 16 couples

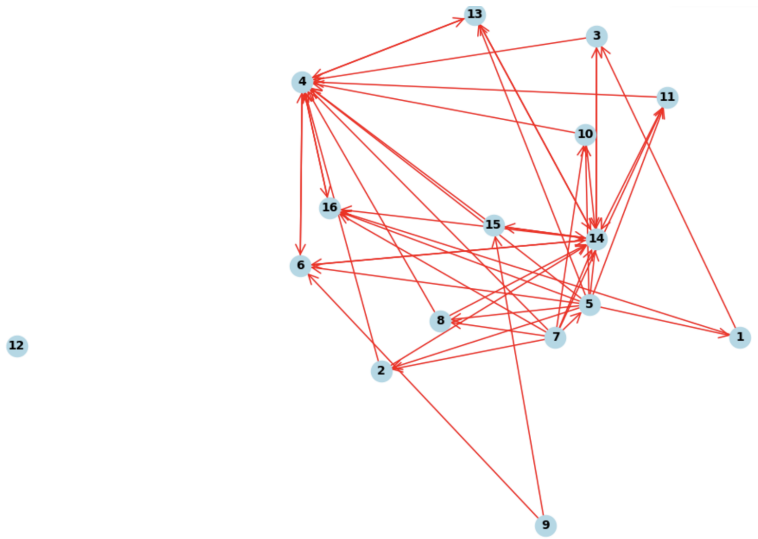


FIGURE 11 – Graphe obtenu avec une instance de 16 couples

8.2 Tableaux obtenus à partir des différentes instances

16 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
1	2	0.4	4	15	4	22	4	17
2	3	0.4	6	16	8	23	6	23
3	2	0.1	2	18	4	24	2	25
4	0	0.2	0	0	0	0	0	0
5	3	0.4	2	20	5	37	2	21
6	4	0.1	2	14	2	19	2	22
7	3	0.3	4	13	5	19	4	15
8	6	0.4	4	13	6	28	4	45
9	7	0.9	8	22	9	34	8	24
10	7	0.4	4	15	4	30	4	26

FIGURE 12 – Tableau des différents résultats obtenus avec une instance de 16 couples

32 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
31	29	3	16	19	23	71	16	28
32	32	1	14	16	16	52	14	29
33	53	2	16	34	20	29	16	27
34	19	2	10	15	18	69	10	17
35	31	8	16	19	22	29	16	20
36	18	2	12	22	14	93	12	28
37	28	2	14	16	17	41	14	23
38	39	11	20	27	23	52	20	21
39	40	3	14	29	20	45	14	29
40	4	1	4	21	4	32	4	19

FIGURE 13 – Tableau des différents résultats obtenus avec une instance de 32 couples

64 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
71	141	44	38	27	47	152	38	38
72	87	13	24	149	39	75	24	32
73	136	176	36	35	43	77	36	43
74	97	49	22	31	36	70	22	21
75	84	24	26	24	33	101	26	23
76	92	60	34	21	44	150	34	21
77	68	20	24	21	34	61	24	26
78	89	44	22	22	33	94	22	23
79	114	43	32	23	39	195	32	37
80	104	70	22	21	30	258	22	23

FIGURE 14 – Tableau des différents résultats obtenus avec une instance de 64 couples

128 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
111	543	246	74	48	83	608	74	126
112	427	442	72	70	83	374	72	71
113	415	198	64	66	78	529	74	69
114	450	219	70	45	84	594	70	50
115	273	671	46	59	65	595	46	57
116	404	207	62	42	73	581	62	42
117	310	737	56	35	70	1099	56	34
118	419	754	70	41	87	679	70	74
119	328	682	66	35	79	793	66	37
120	418	422	68	39	86	596	68	68

FIGURE 15 – Tableau des différents résultats obtenus avec une instance de 128 couples

256 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
151	1842	5000	150	244	166	1000	150	281
152	1726	4600	160	142	175	1300	160	244
153	1779	3000	142	134	159	1200	142	275
154	1677	2800	134	179	145	2200	134	234
155	1855	5700	152	392	168	1400	152	1000

FIGURE 16 – Tableau des différents résultats obtenus avec une instance de 256 couples

512 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
191	7996	70000	338	476				
192	8525	70000	308	570				
193	6346	60000	276	363				
194	7608	60300	276	458				
195	7577	60000	304	483				

FIGURE 17 – Tableau des différents résultats obtenus avec une instance de 512 couples

1024 couples			PLNE		Flot		Matching	
Instance ".wmd"	Cycles	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)	Poids	Temps (ms)
231	31704	1074000	626	1700				
232	30330	990000	606	6249				

FIGURE 18 – Tableau des différents résultats obtenus avec une instance de 1024 couples

8.3 Lien vers nos codes

Tous nos algorithmes sont disponibles via le lien Google Collab suivant : https ://colab.research.google.c
$sUDHkuJHd38jF754QzK2j7l?usp = sharing$

Les fichiers d'instances sont disponibles via le lien suivant : https ://www.preflib.org/dataset/00036

Il est important de noter que les instances que nous utilisons dans nos différents algorithmes sont les instances ".wmd" sans altruistes.